

PRD-14: 后台任务与自动化

版本: 1.0

日期: 2025-12

状态: 已实施 (部分功能)

1. 概述

1.1 文档目的

本文档描述 Blaze Robotics Academy 后台任务与自动化系统的功能需求，包括过期注册处理 (Cron Job)、邮件通知系统、数据同步任务等核心功能。

1.2 核心价值

- 自动化处理**: 减少人工操作，提高效率
- 及时通知**: 自动发送邮件通知，保持用户知情
- 数据一致性**: 定期同步和清理数据，保证系统健康
- 可靠性**: 后台任务稳定运行，确保关键流程不中断

1.3 系统组成

后台任务与自动化系统由以下核心模块组成:

- Cron Jobs**: 定时任务调度和执行
- 邮件通知系统**: 自动发送邮件通知
- 数据同步任务**: 定期同步和清理数据
- 任务监控**: 监控任务执行状态和结果

2. 用户角色与场景

2.1 用户角色

角色	描述	使用场景
系统	后台任务系统	自动执行定时任务
管理员	系统管理员	配置任务、查看任务日志、手动触发任务
用户	注册用户	接收自动邮件通知

2.2 核心使用场景

场景 1: 过期注册处理

1. 系统每天凌晨执行过期注册检查任务
2. 查找所有状态为 `cart` 或 `reserved` 且已过期的注册
3. 将过期注册状态更新为 `expired`
4. 释放课程实例容量
5. 发送过期通知邮件（可选）

场景 2: 课程开始提醒

1. 系统每天检查即将开始的课程
2. 对于未来 24 小时内开始的课程，发送提醒邮件
3. 邮件包含课程信息、时间、地点

场景 3: 等待列表通知

1. 系统检测到课程实例有可用容量
2. 检查等待列表
3. 按顺序通知等待列表中的用户
4. 用户可以在限定时间内报名

场景 4: 课程完成通知

1. 系统检测到课程实例状态变为 `completed`
2. 自动创建用户完成记录
3. 发送课程完成通知邮件
4. 更新学习路径进度

3. 功能需求

3.1 过期注册处理 (Cron Job)

3.1.1 任务描述

- 目的：自动处理过期的购物车和保留状态的注册
- 触发频率：每天凌晨 2:00 AM（可配置）
- 执行逻辑：
 1. 查询所有状态为 `cart` 或 `reserved` 的注册
 2. 检查 `cart_expires_at` 或 `reserved_expires_at` 字段
 3. 如果已过期，更新状态为 `expired`
 4. 释放课程实例容量（减少 `current_students`）
 5. 记录任务执行日志

3.1.2 过期规则

- **Cart 状态：**
 - 默认过期时间：添加到购物车后 30 分钟
 - 可配置：通过环境变量或数据库配置
- **Reserved 状态：**
 - 默认过期时间：保留后 15 分钟
 - 可配置：通过环境变量或数据库配置

3.1.3 容量释放

- 逻辑：
 - 更新 `course_instances.current_students`（减 1）
 - 如果实例从 `is_full = true` 变为 `is_full = false`，触发等待列表通知
- 事务：
 - 使用数据库事务确保数据一致性
 - 失败时回滚

3.1.4 通知（可选）

- 功能：
 - 发送过期通知邮件

- 提醒用户注册已过期
 - 提供重新报名的链接
- 配置：
 - 可通过配置开启/关闭
 - 避免过度通知

3.2 邮件通知系统

3.2.1 课程开始提醒

- 触发条件：
 - 课程实例开始时间在未来 24 小时内
 - 用户已报名（状态为 `enrolled`）
- 邮件内容：
 - 课程名称
 - 开始时间和地点
 - 课程描述
 - 准备事项（如果有）
 - 取消链接（如果需要）
- 发送时间：
 - 每天上午 9:00 AM 检查并发送

3.2.2 等待列表通知

- 触发条件：
 - 课程实例有可用容量（`is_full = false`）
 - 等待列表中有用户
- 邮件内容：
 - 课程名称和时间
 - 可用容量信息
 - 报名链接（带 token，24 小时有效）
 - 过期时间提醒
- 发送逻辑：
 - 按等待列表顺序发送
 - 每次只通知前 N 个用户（如 5 个）
 - 等待用户报名或过期后再通知下一批

3.2.3 课程完成通知

- 触发条件：
 - 课程实例状态变为 `completed`
 - 用户已完成课程
- 邮件内容：
 - 课程名称
 - 完成日期
 - 学习成果总结
 - 证书链接（如果有）
 - 推荐下一步课程

3.2.4 注册确认通知

- 触发条件：
 - 用户成功报名课程（状态变为 `enrolled`）
- 邮件内容：
 - 课程名称和时间
 - 地点信息
 - 支付确认（如果已支付）
 - 课程准备事项
 - 取消政策

3.2.5 支付提醒

- 触发条件：
 - 注册状态为 `reserved`，即将过期
 - 用户尚未完成支付
- 邮件内容：
 - 课程信息
 - 支付链接
 - 过期时间提醒
 - 紧急提示

3.3 数据同步任务

3.3.1 学习路径进度同步

- 目的：定期更新用户学习路径进度
- 触发频率：每小时
- 执行逻辑：

1. 查询所有活跃的学习路径
2. 对于每个路径，计算所有用户的进度
3. 更新 `user_learning_path_progress` 表
4. 检测路径完成情况

3.3.2 课程实例状态同步

- 目的：自动更新课程实例状态
- 触发频率：每天
- 执行逻辑：
 1. 查询所有 `scheduled` 状态的实例
 2. 如果 `start_date` 已到，更新为 `ongoing`
 3. 查询所有 `ongoing` 状态的实例
 4. 如果 `end_date` 已过，更新为 `completed`
 5. 触发课程完成通知

3.3.3 数据清理任务

- 目的：清理过期和无效数据
- 触发频率：每周
- 清理内容：
 - 删除过期的 `expired` 状态注册（超过 90 天）
 - 清理临时文件
 - 清理过期的会话数据

3.4 任务监控

3.4.1 任务执行日志

- 记录内容：
 - 任务名称
 - 执行时间
 - 执行结果（成功/失败）
 - 处理记录数
 - 错误信息（如果有）
 - 执行耗时
- 存储：
 - 数据库表： `cron_job_logs`
 - 保留最近 90 天的日志

3.4.2 任务状态监控

- 监控指标：
 - 任务执行成功率
 - 平均执行时间
 - 失败任务列表
 - 最近执行时间
- 告警：
 - 任务连续失败 3 次发送告警
 - 任务执行时间超过阈值发送告警

3.4.3 任务管理界面（未来）

- 位置： `/admin/cron-jobs`
 - 功能：
 - 查看所有任务列表
 - 查看任务执行历史
 - 手动触发任务
 - 配置任务参数
 - 启用/禁用任务
-

4. 数据模型

4.1 核心数据表

4.1.1 cron_job_logs（Cron 任务日志表）

- `id`：日志 ID
- `job_name`：任务名称
- `status`：状态（success, failed）
- `started_at`：开始时间
- `completed_at`：完成时间
- `duration_ms`：执行耗时（毫秒）
- `records_processed`：处理记录数
- `error_message`：错误信息（如果有）
- `metadata`：元数据（JSON）

4.1.2 email_notifications (邮件通知表, 可选)

- `id`: 通知 ID
 - `user_id`: 用户 ID
 - `type`: 通知类型 (course_start, waitlist, completion等)
 - `subject`: 邮件主题
 - `content`: 邮件内容
 - `sent_at`: 发送时间
 - `status`: 状态 (pending, sent, failed)
 - `error_message`: 错误信息 (如果有)
-

5. API 设计

5.1 Cron Job API

5.1.1 执行过期注册处理

```
POST /api/cron/process-expired-enrollments
```

Headers:

```
- Authorization: Bearer {cron_secret}
```

Response:

```
{
  "success": boolean,
  "processed": number,
  "expired": number,
  "errors": number,
  "duration_ms": number
}
```

5.1.2 执行课程开始提醒

```
POST /api/cron/send-course-reminders
```

Headers:

```
- Authorization: Bearer {cron_secret}
```

Response:

```
{
  "success": boolean,
  "sent": number,
  "errors": number,
  "duration_ms": number
}
```

5.1.3 执行等待列表通知

POST /api/cron/notify-waitlist

Headers:

– Authorization: Bearer {cron_secret}

Response:

```
{
  "success": boolean,
  "notified": number,
  "errors": number,
  "duration_ms": number
}
```

5.2 任务管理 API（未来）

5.2.1 获取任务列表

GET /api/admin/cron-jobs

Response:

```
{
  "jobs": CronJob[],
  "stats": {
    "total": number,
    "enabled": number,
    "disabled": number
  }
}
```

5.2.2 手动触发任务

POST /api/admin/cron-jobs/[name]/trigger

Response:

```
{
  "success": boolean,
  "job_id": string,
  "started_at": string
}
```

6. 技术实现

6.1 Cron Job 调度

6.1.1 Vercel Cron Jobs

- 配置: `vercel.json`
- 格式:

```
{
  "crons": [
    {
      "path": "/api/cron/process-expired-enrollments",
      "schedule": "0 2 * * *"
    }
  ]
}
```

- 优势:
 - 无需额外服务
 - 自动扩展
 - 免费额度充足

6.1.2 外部 Cron 服务（备选）

- 选项:
 - GitHub Actions（免费）
 - EasyCron（付费）
 - Cron-job.org（免费）
- 实现:
 - 定期调用 API 端点

- 使用 Bearer Token 认证

6.2 邮件服务

6.2.1 SMTP 配置

- 服务商：
 - SendGrid (推荐)
 - AWS SES
 - Mailgun
 - Nodemailer (直接 SMTP)
- 配置：
 - 环境变量存储 SMTP 配置
 - 支持多个邮件服务商

6.2.2 邮件模板

- 技术：React Email 或 HTML 模板
- 内容：
 - 品牌 Logo 和颜色
 - 响应式设计
 - 清晰的 CTA 按钮
- 多语言：支持中英文 (未来)

6.3 任务安全

6.3.1 认证机制

- **Bearer Token**：
 - 使用环境变量 `CRON_SECRET`
 - 所有 Cron API 需要验证
- **IP 白名单 (可选)**：
 - 限制只有特定 IP 可以调用

6.3.2 错误处理

- **重试机制**：
 - 失败任务自动重试 (最多 3 次)
 - 使用指数退避

- 告警：
 - 连续失败发送告警邮件
 - 记录到监控系统
-

7. 业务规则

7.1 过期处理规则

- Cart 状态：30 分钟后过期
- Reserved 状态：15 分钟后过期
- 过期后自动释放容量
- 过期注册不能恢复

7.2 通知规则

- 同一用户同一类型通知 24 小时内只发送一次
- 等待列表通知按顺序发送
- 邮件发送失败记录日志，不阻塞主流程

7.3 任务执行规则

- 任务执行时间限制：最长 10 分钟
 - 失败任务自动重试（最多 3 次）
 - 任务执行结果记录日志
-

8. 非功能需求

8.1 性能要求

- 任务执行时间 < 5 分钟（大部分任务）
- 邮件发送成功率 > 95%
- 任务执行不阻塞主系统

8.2 可靠性要求

- 任务执行成功率 > 99%
- 失败任务自动重试
- 关键任务有监控和告警

8.3 可维护性要求

- 任务配置可调整
 - 任务日志完整
 - 任务状态可查询
-

9. 验收标准

9.1 功能验收

- ☒ 过期注册自动处理
- ☒ 邮件通知正确发送
- ☒ 数据同步任务正常执行
- ☒ 任务日志完整记录

9.2 性能验收

- ☒ 任务执行时间符合要求
 - ☒ 邮件发送成功率符合要求
 - ☒ 系统负载正常
-

10. 实施状态

10.1 已完成功能 (✅)

- ☒ 过期注册处理 (基本实现)
- ☒ 邮件通知系统 (基本实现)
- ☒ Cron Job 配置 (Vercel Cron)

10.2 部分完成功能 (⚠️)

- ⚠️ 课程开始提醒（待完善）
- ⚠️ 等待列表通知（待完善）
- ⚠️ 任务监控（基本日志已实现，界面待开发）

10.3 计划功能（🕒）

- 🕒 任务管理界面
 - 🕒 高级任务调度
 - 🕒 任务性能分析
 - 🕒 邮件模板管理
-

11. 未来扩展

11.1 短期（1-3 个月）

- 🕒 完善所有邮件通知类型
- 🕒 任务管理界面
- 🕒 任务性能监控

11.2 中期（3-6 个月）

- 🕒 高级任务调度（条件触发、依赖任务）
- 🕒 任务队列系统
- 🕒 邮件模板可视化编辑

11.3 长期（6-12 个月）

- 🕒 分布式任务调度
 - 🕒 任务工作流引擎
 - 🕒 AI 驱动的任务优化
-

12. 附录

12.1 相关文档

- `CRON_SETUP.md` : Cron Job 设置指南
- `SMTP_SETUP_GUIDE.md` : SMTP 邮件设置指南
- `SMTP_TROUBLESHOOTING.md` : SMTP 问题排查

12.2 术语表

- **Cron Job (定时任务)**: 按计划自动执行的任务
- **Cron Expression (Cron 表达式)**: 定义任务执行时间的表达式
- **SMTP (Simple Mail Transfer Protocol)**: 邮件传输协议
- **Bearer Token (Bearer 令牌)**: 用于 API 认证的令牌